

MLC基本思想

目录

- 机器学习编译基本思想
 - 什么是机器学习编译
 - TVM中的一些概念及编译的思路
 - IRModule 和 runtime::Module
 - TensorIR 与 Schedule 变换
 - 端到端模型执行
 - 图优化 FuseOps
 - 调用外部库
 - 总结
 - References
- Inference与CINN结合进行推理
 - CINN目前相对缺失的工作（了解有限）
 - 思路一：基于CINN做推理
 - 思路二：编译器只用来做代码生成
 - 思路三：编译器作为子图引擎

机器学习编译基本思想

什么是机器学习编译

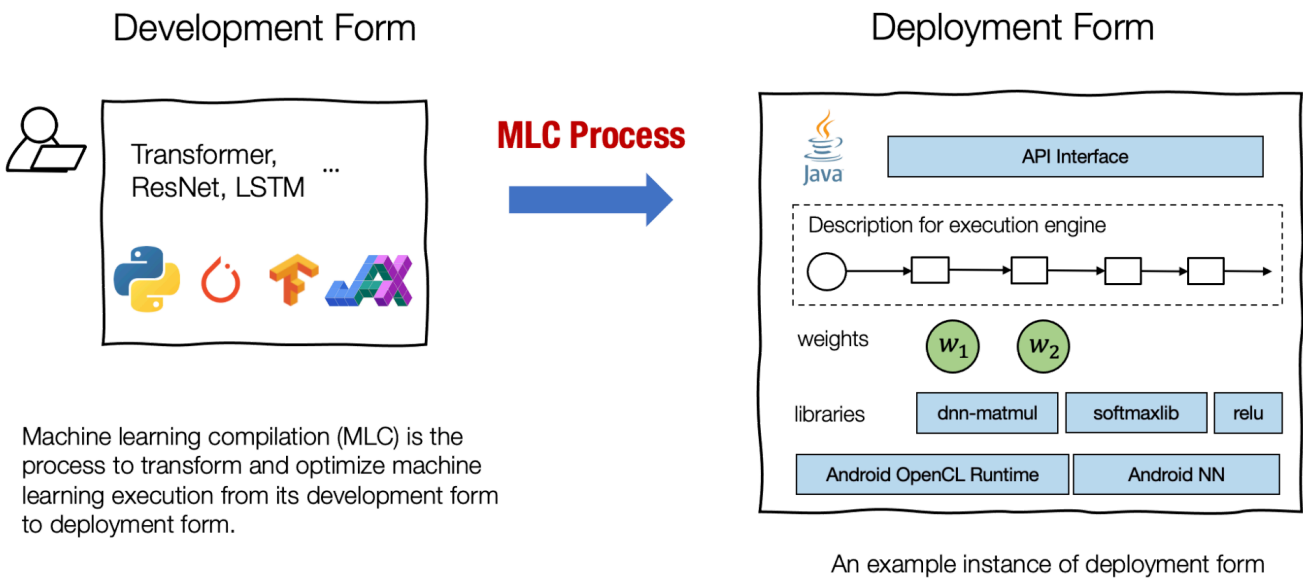
mlc.ai 中对机器学习编译的定义如下：

机器学习编译 (machine learning compilation, MLC) 是指，将机器学习算法从开发阶段，通过变换和优化算法，使其变成部署状态。

开发形式是指我们在开发机器学习模型时使用的形式。典型的开发形式包括用 Paddle、PyTorch、TensorFlow 或 JAX 等通用框架编写的模型描述，以及与之相关的权重。

部署形式是指执行机器学习应用程序所需的形式。它通常涉及机器学习模型的每个步骤的支撑代码、管理资源（例如内存）的控制器，以及与应用程序开发环境的接口（例如用于 android 应用程序的 java API）。

（摘抄自 mlc 课件）

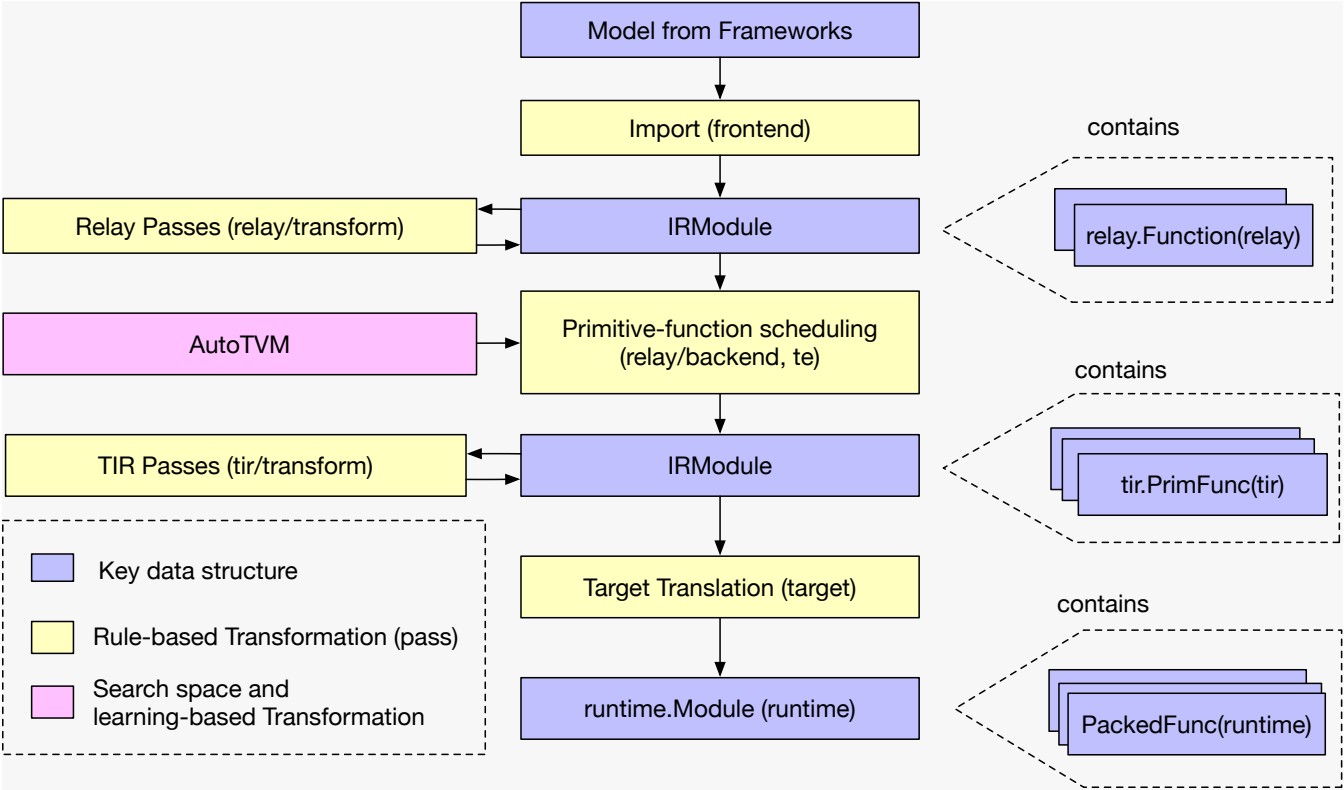


机器学习编译的目的：

- 集成与最小化依赖 部署过程通常涉及集成 (Integration)，即将必要的元素组合在一起以用于部署应用程序。例如，如果我们想启用一个安卓相机应用程序来检测猫，我们将需要图像分类模型的必要代码，但不需要模型无关的其他部分（例如，我们不需要包括用于 NLP 应用程序的 embedding table）。代码集成、最小化依赖项的能力能够减小应用的大小，并且可以使应用程序部署到的更多的环境。
- 利用硬件加速 每个部署环境都有自己的一套原生加速技术，并且其中许多是专门为机器学习开发的。机器学习编译的一个目标就是利用硬件本身的特性进行加速。我们可以通过构建调用原生加速库的部署代码或生成利用原生指令（如 TensorCore）的代码来做到这一点。
- 通用优化 有许多等效的方法可以运行相同的模型执行。MLC 的通用优化形式是不同形式的优化，以最小化内存使用或提高执行效率的方式转换模型执行。

(摘抄自 mlc 课件)

TVM中的一些概念及编译的思路



上图是 tvml 的编译流程，外部的模型（paddle、pytorch等）导入得到 relay/relax 的高层 ir 表示，进一步 lower 后得到底层 tensor ir 表示，进一步编译可得到可执行的机器码；

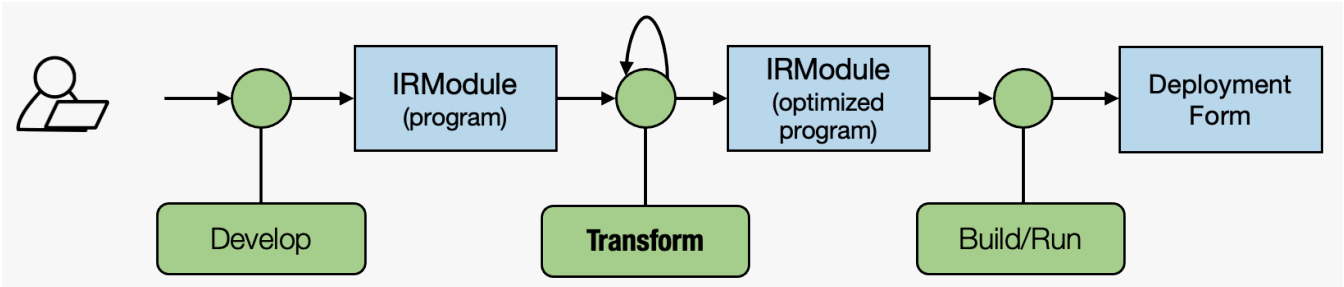
IRModule 和 runtime::Module

IRModule 是程序变换的核心组件，其内部hold了一系列 Function（可以是relay/relax::Function 也可以是 tir::PrimFunc）；

</> C++ | 收起 ^

```
1 class IRModuleNode : public Object {
2   public:
3     /*! \brief A map from ids to all global functions. */
4     Map<GlobalVar, BaseFunc> functions;
5     ...
}
```

机器学习编译的基本思想就是变换：对于相同的计算逻辑进行等价的变换，使其访存更加友好、尽可能的适配底层硬件的特殊指令集等；



`runtime::Module` 是 `IRModule` 的编译产物，可接受输入、运行、返回输出，类似于执行器；（目前不是我们关注的重点）

</>

Python | 收起 ^

```
1 ir_module = ...
2 with tvm.transform.PassContext(opt_level=3):
3     rt_mod = tvm.build(ir_module, target)
4 output = rt_mod['main'](input, ...)
```

TensorIR 与 Schedule 变换

TensorIR 是 TVM 中针对 tensor 的计算过程的描述（DSL）

```
from tvm.script import tir as T
```

```
@T.prim_func
```

```
def main(A: T.Buffer[128, "float32"],
         B: T.Buffer[128, "float32"],
         C: T.Buffer[128, "float32"]):
```

(Multi-dimensional) buffers that holds the input, output, and intermediate results.

```
for i in range(128):
    with T.block("C"):
```

Loop nests that drive compute iterations.

```
        vi = T.axis.spatial(128, i)
        C[vi] = A[vi] + B[vi]
```

Computations statements.

再看一个 `matmul + relu` 的 TensorIR 表示和 `numpy` 的对比（可以一一对应）

```
@tvm.script.ir_module
class MyModule:
    @T.prim_func
    def mm_relu(A: T.Buffer[(128, 128), "float32"],
               B: T.Buffer[(128, 128), "float32"],
               C: T.Buffer[(128, 128), "float32"]):
        T.func_attr({"global_symbol": "mm_relu", "tir.noalias": True})
        Y = T.alloc_buffer((128, 128), dtype="float32")
        for i, j, k in T.grid(128, 128, 128):
            with T.block("Y"):
                vi = T.axis.spatial(128, i)
                vj = T.axis.spatial(128, j)
                vk = T.axis.reduce(128, k)
                with T.init():
                    Y[vi, vj] = T.float32(0)
                Y[vi, vj] = Y[vi, vj] + A[vi, vk] * B[vk, vj]
        for i, j in T.grid(128, 128):
            with T.block("C"):
                vi = T.axis.spatial(128, i)
                vj = T.axis.spatial(128, j)
                C[vi, vj] = T.max(Y[vi, vj], T.float32(0))
```

```
def numpy_mm_relu(A: np.ndarray, B: np.ndarray, C: np.ndarray):
    Y = np.empty((128, 128), dtype="float32")
    for i in range(128):
        for j in range(128):
            for k in range(128):
                if k == 0:
                    Y[i, j] = 0
                Y[i, j] = Y[i, j] + A[i, k] * B[k, j]
    for i in range(128):
        for j in range(128):
            C[i, j] = max(Y[i, j], 0)
```

Schedule 程序变换的基本组件，tvm 提供了多种方法，可参考[官方文档](#)

举例，针对上述的 `matmul + relu` 进行变换

原始的 `IRModule` 如下：

</>

Python | 收起 ^

```

1 @tvm.script.ir_module
2 class Module:
3     @T.prim_func
4     def mm_relu(A: T.Buffer[(128, 128), "float32"], B: T.Buffer[(128,
5         128), "float32"], C: T.Buffer[(128, 128), "float32"]) -> None:
6         # function attr dict
7         T.func_attr({"global_symbol": "mm_relu", "tir.noalias": True})
8         # body
9         # with T.block("root")
10        Y = T.alloc_buffer([128, 128], dtype="float32")
11        for i, j, k in T.grid(128, 128, 128):
12            with T.block("Y"):
13                vi, vj, vk = T.axis.remap("SSR", [i, j, k])
14                T.reads(A[vi, vk], B[vk, vj])
15                T.writes(Y[vi, vj])
16                with T.init():
17                    Y[vi, vj] = T.float32(0)
18                Y[vi, vj] = Y[vi, vj] + A[vi, vk] * B[vk, vj]
19        for i, j in T.grid(128, 128):
20            with T.block("C"):
21                vi, vj = T.axis.remap("SS", [i, j])
22                T.reads(Y[vi, vj])
23                T.writes(C[vi, vj])
24                C[vi, vj] = T.max(Y[vi, vj], T.float32(0))

```

进行 split 变换

</>

Python | 收起 ^

```

1 sch = tvm.tir.Schedule(MyModule)
2 block_Y = sch.get_block("Y", func_name="mm_relu")
3 i, j, k = sch.get_loops(block_Y)
4 j0, j1 = sch.split(j, factors=[None, 4])

```

变换后的 IRModule 如下:

</>

Python | 收起 ^

```

1 @tvm.script.ir_module
2 class Module:
3     @T.prim_func
4     def mm_relu(A: T.Buffer[(128, 128), "float32"], B: T.Buffer[(128,
5         128), "float32"], C: T.Buffer[(128, 128), "float32"]) -> None:
6         # function attr dict

```

```

6      T.func_attr({"global_symbol": "mm_relu", "tir.noalias": True})
7      # body
8      # with T.block("root")
9      Y = T.alloc_buffer([128, 128], dtype="float32")
10     for i, j_0, j_1, k in T.grid(128, 32, 4, 128):
11         with T.block("Y"):
12             vi = T.axis.spatial(128, i)
13             vj = T.axis.spatial(128, j_0 * 4 + j_1)
14             vk = T.axis.reduce(128, k)
15             T.reads(A[vi, vk], B[vk, vj])
16             T.writes(Y[vi, vj])
17             with T.init():
18                 Y[vi, vj] = T.float32(0)
19             Y[vi, vj] = Y[vi, vj] + A[vi, vk] * B[vk, vj]
20     for i, j in T.grid(128, 128):
21         with T.block("C"):
22             vi, vj = T.axis.remap("SS", [i, j])
23             T.reads(Y[vi, vj])
24             T.writes(C[vi, vj])
25             C[vi, vj] = T.max(Y[vi, vj], T.float32(0))

```

reorder 变换:

</>

Python | 收起 ^

```
1 sch.reorder(j0, k, j1)
```

</>

Python | 收起 ^

```

1 @tvm.script.ir_module
2 class Module:
3     @T.prim_func
4     def mm_relu(A: T.Buffer[(128, 128), "float32"], B: T.Buffer[(128,
5     128), "float32"], C: T.Buffer[(128, 128), "float32"]) -> None:
6         # function attr dict
7         T.func_attr({"global_symbol": "mm_relu", "tir.noalias": True})
8         # body
9         # with T.block("root")
10        Y = T.alloc_buffer([128, 128], dtype="float32")
11        for i, j_0, k, j_1 in T.grid(128, 32, 128, 4):
12            with T.block("Y"):
13                vi = T.axis.spatial(128, i)
14                vj = T.axis.spatial(128, j_0 * 4 + j_1)
15                vk = T.axis.reduce(128, k)

```

```

15         T.reads(A[vi, vk], B[vk, vj])
16         T.writes(Y[vi, vj])
17         with T.init():
18             Y[vi, vj] = T.float32(0)
19             Y[vi, vj] = Y[vi, vj] + A[vi, vk] * B[vk, vj]
20     for i, j in T.grid(128, 128):
21         with T.block("C"):
22             vi, vj = T.axis.remap("SS", [i, j])
23             T.reads(Y[vi, vj])
24             T.writes(C[vi, vj])
25             C[vi, vj] = T.max(Y[vi, vj], T.float32(0))

```

reverse_compute_at 变换:

</>

Plain Text | 收起 ^

```

1 block_C = sch.get_block("C", "mm_relu")
2 sch.reverse_compute_at(block_C, j0)

```

</>

Python | 收起 ^

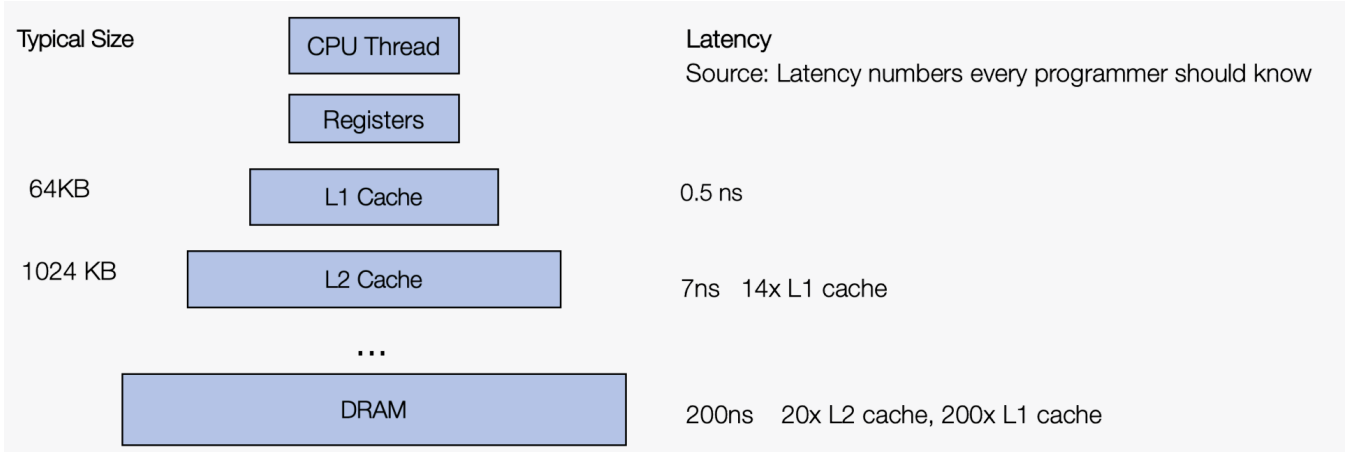
```

1 @tvm.script.ir_module
2 class Module:
3     @T.prim_func
4     def mm_relu(A: T.Buffer[(128, 128), "float32"], B: T.Buffer[(128,
5     128), "float32"], C: T.Buffer[(128, 128), "float32"]) -> None:
6         # function attr dict
7         T.func_attr({"global_symbol": "mm_relu", "tir.noalias": True})
8         # body
9         # with T.block("root")
10        Y = T.alloc_buffer([128, 128], dtype="float32")
11        for i, j_0 in T.grid(128, 32):
12            for k, j_1 in T.grid(128, 4):
13                with T.block("Y"):
14                    vi = T.axis.spatial(128, i)
15                    vj = T.axis.spatial(128, j_0 * 4 + j_1)
16                    vk = T.axis.reduce(128, k)
17                    T.reads(A[vi, vk], B[vk, vj])
18                    T.writes(Y[vi, vj])
19                    with T.init():
20                        Y[vi, vj] = T.float32(0)
21                        Y[vi, vj] = Y[vi, vj] + A[vi, vk] * B[vk, vj]
22        for ax0 in T.serial(4):
23            with T.block("C"):

```

```
23         vi = T.axis.spatial(128, i)
24         vj = T.axis.spatial(128, j_0 * 4 + ax0)
25         T.reads(Y[vi, vj])
26         T.writes(C[vi, vj])
27         C[vi, vj] = T.max(Y[vi, vj], T.float32(0))
```

以上都是对 IRModule 的变换，对相同计算的不同描述；现有计算机体系结构的原因，通过程序变换，使其访存更加友好，整体性能也会更好。



要了解为什么不同的循环变体会导致不同的性能，我们需要回顾一个事实，即访问 A 和 B 中的任何内存块的速度并不一致。现代 CPU 带有多级缓存，需要先将数据提取到缓存中，然后 CPU 才能访问它。

重要的是，访问已经在缓存中的数据要快得多。CPU 采用的一种策略是获取彼此更接近的数据。当我们读取内存中的一个元素时，它会尝试将附近的元素（更正式的名称为“缓存行”）获取到缓存中。因此，当你读取下一个元素时，它已经在缓存中。因此，具有连续内存访问的代码通常比随机访问内存不同部分的代码更快。

（摘抄自mlc课件）

上述是人工进行 schedule 变换的例子，也可以通过自动程序优化的方式（搜索空间 + 搜索方法）进行性能优化，TVM 中称为 **Meta-Schedule**；针对128x128x128的矩阵乘，启用自动程序优化，看下搜索得到的一条优化路径。（相比初始版本性能优化很多，一定程度上能和专家优化的工业级库进行对比）。

</> Plain Text | 收起 ^

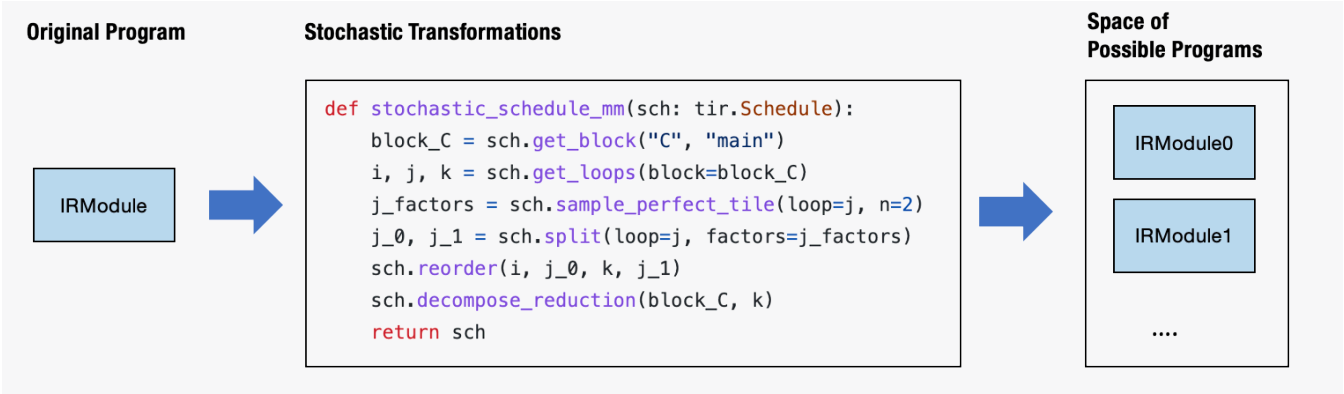
```
1 b0 = sch.get_block(name="C", func_name="main")
2 b1 = sch.get_block(name="root", func_name="main")
3 sch.annotate(block_or_loop=b0, ann_key="meta_schedule.tiling_structure",
4               ann_val="SSRSRS")
5 l2, l3, l4 = sch.get_loops(block=b0)
6 v5, v6, v7, v8 = sch.sample_perfect_tile(loop=l2, n=4,
7      max_innermost_factor=64, decision=[4, 1, 32, 1])
```



```
6 l9, l10, l11, l12 = sch.split(loop=l2, factors=[v5, v6, v7, v8],
  preserve_unit_iters=True)
7 v13, v14, v15, v16 = sch.sample_perfect_tile(loop=l3, n=4,
  max_innermost_factor=64, decision=[4, 1, 8, 4])
8 l17, l18, l19, l20 = sch.split(loop=l3, factors=[v13, v14, v15, v16],
  preserve_unit_iters=True)
9 v21, v22 = sch.sample_perfect_tile(loop=l4, n=2,
  max_innermost_factor=64, decision=[64, 2])
10 l23, l24 = sch.split(loop=l4, factors=[v21, v22],
  preserve_unit_iters=True)
11 sch.reorder(l9, l17, l10, l18, l23, l11, l19, l24, l12, l20)
12 b25 = sch.cache_write(block=b0, write_buffer_index=0,
  storage_scope="global")
13 sch.reverse_compute_at(block=b25, loop=l18, preserve_unit_loops=True)
14 sch.annotate(block_or_loop=b1, ann_key="meta_schedule.parallel",
  ann_val=16)
15 sch.annotate(block_or_loop=b1, ann_key="meta_schedule.vectorize",
  ann_val=64)
16 v26 = sch.sample_categorical(candidates=[0, 16, 64, 512], probs=[0.25,
  0.25, 0.25, 0.25], decision=1)
17 sch.annotate(block_or_loop=b1, ann_key="meta_schedule.unroll_explicit",
  ann_val=v26)
18 sch.enter_postproc()
19 b27 = sch.get_block(name="root", func_name="main")
20 sch.unannotate(block_or_loop=b27, ann_key="meta_schedule.parallel")
21 sch.unannotate(block_or_loop=b27, ann_key="meta_schedule.vectorize")
22 sch.unannotate(block_or_loop=b27,
  ann_key="meta_schedule.unroll_explicit")
23 b28, b29 = sch.get_child_blocks(b27)
24 l30, l31, l32, l33, l34, l35, l36, l37, l38, l39 =
  sch.get_loops(block=b28)
25 l40 = sch.fuse(l30, l31, l32, l33, preserve_unit_iters=True)
26 sch.parallel(loop=l40)
27 l41 = sch.fuse(l39, preserve_unit_iters=True)
28 sch.vectorize(loop=l41)
29 sch.annotate(block_or_loop=l40, ann_key="pragma_auto_unroll_max_step",
  ann_val=16)
30 sch.annotate(block_or_loop=l40, ann_key="pragma_unroll_explicit",
  ann_val=1)
31 l42, l43, l44 = sch.get_loops(block=b29)
32 l45 = sch.fuse(l44, preserve_unit_iters=True)
33 sch.vectorize(loop=l45)
34 sch.annotate(block_or_loop=l42, ann_key="pragma_auto_unroll_max_step",
  ann_val=16)
```

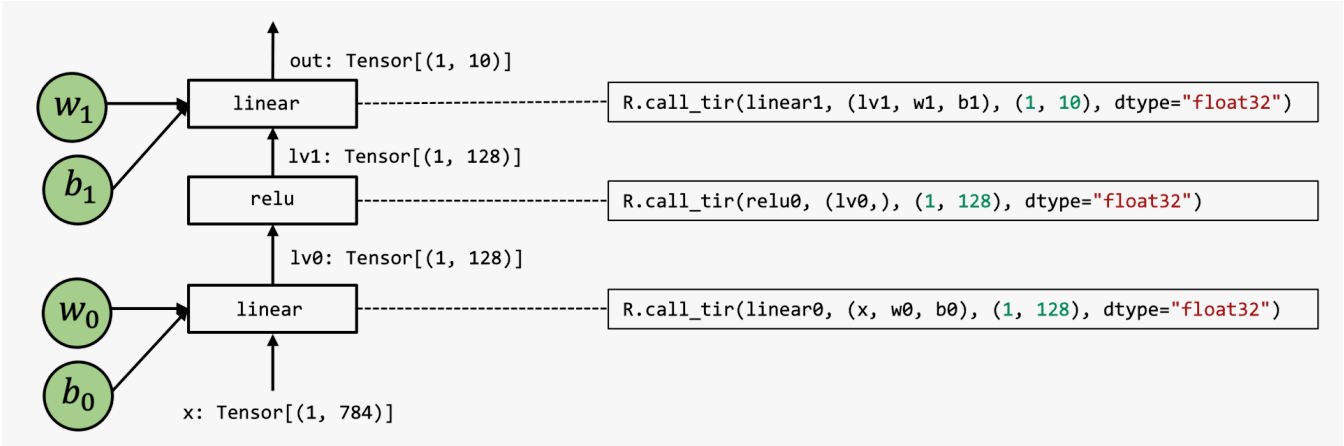
```
35 sch.annotate(block_or_loop=l42, ann_key="pragma_unroll_explicit",
    ann_val=1)
36 b46 = sch.get_block(name="C", func_name="main")
37 l47, l48, l49, l50, l51, l52, l53 = sch.get_loops(block=b46)
38 b54 = sch.decompose_reduction(block=b46, loop=l48)
```

通过 **Schedule** 变换的方式，得到原始计算的多种表示，在这多组表示中选择性能最好的，实现了对 **TensorIR** 加速的目的



端到端模型执行

上面介绍了 TensorIR 及其变换方法，在模型部署过程中常常会涉及多个算子，如下图所示，涉及到了3个 TensorIR：linear0, linear1 和 relu0。



TVM 通过高层 IR 描述了整个模型的执行过程：

</> Python | 收起 ^

```
1 @tvm.script.ir_module
2 class MyModule:
3     @T.prim_func
4     def relu0(X: T.Buffer[(1, 128), "float32"],
5             Y: T.Buffer[(1, 128), "float32"]):
6         ...
```

```

7
8 @T.prim_func
9 def dense0(X: T.Buffer[(1, 784), "float32"],
10           W: T.Buffer[(128, 784), "float32"],
11           Z: T.Buffer[(1, 128), "float32"]):
12     ...
13
14 @T.prim_func
15 def add0(X: T.Buffer[(1, 128), "float32"],
16         B: T.Buffer[(128,), "float32"],
17         Z: T.Buffer[(1, 128), "float32"]):
18     ...
19
20
21 @T.prim_func
22 def dense1(X: T.Buffer[(1, 128), "float32"],
23           W: T.Buffer[(10, 128), "float32"],
24           Z: T.Buffer[(1, 10), "float32"]):
25     ...
26
27 @T.prim_func
28 def add1(X: T.Buffer[(1, 10), "float32"],
29         B: T.Buffer[(10,), "float32"],
30         Z: T.Buffer[(1, 10), "float32"]):
31     ...
32
33 @R.function
34 def main(x: Tensor((1, 784), "float32"),
35         w0: Tensor((128, 784), "float32"),
36         b0: Tensor((128,), "float32"),
37         w1: Tensor((10, 128), "float32"),
38         b1: Tensor((10,), "float32")):
39     with R.dataflow():
40         lv0 = R.call_tir(dense0, (x, w0), (1, 128), dtype="float32")
41         lv1 = R.call_tir(add0, (lv0, b0), (1, 128), dtype="float32")
42         lv2 = R.call_tir(relu0, (lv1,), (1, 128), dtype="float32")
43         lv3 = R.call_tir(dense1, (lv2, w1), (1, 10),
dtype="float32")
44         out = R.call_tir(add1, (lv3, b1), (1, 10), dtype="float32")
45         R.output(out)
46     return out

```

图优化 FuseOps

TVM 支持算子自动融合，融合后 IRModule 如下（此处举例没有融合relu，实际代码执行会融合）。可以看到 `fused_dense_add0` 和 `fused_dense_add1` 仍然是上层 relax/relay 函数，它们调用相应的 TensorIR dense 和 add 函数。

</>

Python | 收起 ^

```

1 @tvm.script.ir_module
2 class Module:
3     @R.function
4     def fused_dense_add1(x: Tensor((1, 128), "float32"), w: Tensor((10,
5         128), "float32"), b: Tensor((10,), "float32")) -> Tensor(None,
6         "float32", ndim = 2):
7         ...
8
9     @T.prim_func
10    def relu(rxplaceholder: T.Buffer[(1, T.int64(128))], "float32",
11        compute: T.Buffer[(1, T.int64(128))], "float32"]) -> None:
12        ...
13
14    @T.prim_func
15    def dense1(rxplaceholder: T.Buffer[(1, T.int64(128))], "float32",
16        rxplaceholder_1: T.Buffer[(T.int64(10), T.int64(128))], "float32",
17        T_matmul_NT: T.Buffer[(1, T.int64(10))], "float32"]) -> None:
18        ...
19
20    @T.prim_func
21    def add1(rxplaceholder: T.Buffer[(1, T.int64(10))], "float32",
22        rxplaceholder_1: T.Buffer[(T.int64(10), "float32"), T_add: T.Buffer[(1,
23            T.int64(10))], "float32"]) -> None:
24        ...
25
26    @T.prim_func
27    def dense(rxplaceholder: T.Buffer[(1, 784), "float32"],
28        rxplaceholder_1: T.Buffer[(T.int64(128), T.int64(784))], "float32",
29        T_matmul_NT: T.Buffer[(1, T.int64(128))], "float32"]) -> None:
30        ...
31
32    @T.prim_func
33    def add(rxplaceholder: T.Buffer[(1, T.int64(128))], "float32",
34        rxplaceholder_1: T.Buffer[(T.int64(128), "float32"), T_add: T.Buffer[(1,
35            T.int64(128))], "float32"]) -> None:
36        ...
37
38    @R.function

```

```

28     def fused_dense_add0(x1: Tensor((1, 784), "float32"), w1:
Tensor((128, 784), "float32"), b1: Tensor((128,), "float32")) ->
Tensor(None, "float32", ndim = 2):
29         # block 0
30         with R.dataflow():
31             lv1 = R.call_tir(dense, (x1, w1), (1, 128), dtype="float32")
32             gv1 = R.call_tir(add, (lv1, b1), (1, 128), dtype="float32")
33             R.output(gv1)
34         return gv1
35
36     @R.function
37     def main(x2: Tensor((1, 784), "float32")) -> Tensor(None, "float32",
ndim = 2):
38         # block 0
39         with R.dataflow():
40             lv11: Tensor((1, 128), "float32") = fused_dense_add0(x2,
meta[relay.Constant][0], meta[relay.Constant][1])
41             lv2 = R.call_tir(relu, (lv11,), (1, 128), dtype="float32")
42             lv4: Tensor((1, 10), "float32") = fused_dense_add1(lv2,
meta[relay.Constant][2], meta[relay.Constant][3])
43             gv2: Tensor((1, 10), "float32") = lv4
44             R.output(gv2)
45         return gv2

```

此时的融合实际上只发生在高层 IR 表示上，从底层 IR 的角度去看实际上是没有融合的。还需要额外的工作（FuseTIR）将它们变成一个单一的 TensorIR 函数；实际的 Lower 过程中会跑执行这个逻辑，relax IR 中负责这块处理的逻辑如下：

</>

Python | 收起 ^

```
1 MLPModelFinal = relax.transform.FuseTIR()(MLPModelTIR)
```

Lower后的 IRModule 如下，可以看出 TensorIR 层面已经完成了融合：

</>

Python | 收起 ^

```

1 @tvm.script.ir_module
2 class Module:
3     @T.prim_func
4     def fused_dense_add0(x: T.Buffer[(1, 784), "float32"], w:
T.Buffer[(T.int64(128), T.int64(784)), "float32"], b:
T.Buffer[T.int64(128), "float32"], T_add: T.Buffer[(1, T.int64(128)),
"float32"]) -> None:
5         # function attr dict

```

```

6      T.func_attr({"tir.noalias": True, "global_symbol":
"fused_dense_add0"})
7      # body
8      # with T.block("root")
9      T_matmul_NT = T.alloc_buffer([1, T.int64(128)], dtype="float32")
10     for i0, i1, i2 in T.grid(1, T.int64(128), 784):
11         with T.block("T_matmul_NT"):
12             i = T.axis.spatial(1, i0)
13             j = T.axis.spatial(T.int64(128), i1)
14             k = T.axis.reduce(784, i2)
15             T.reads(x[i, k], w[j, k])
16             T.writes(T_matmul_NT[i, j])
17             with T.init():
18                 T_matmul_NT[i, j] = T.float32(0)
19             T_matmul_NT[i, j] = T_matmul_NT[i, j] + x[i, k] * w[j,
k]
20     for i0, i1 in T.grid(1, T.int64(128)):
21         with T.block("T_add"):
22             ax0 = T.axis.spatial(1, i0)
23             ax1 = T.axis.spatial(T.int64(128), i1)
24             T.reads(T_matmul_NT[ax0, ax1], b[ax1])
25             T.writes(T_add[ax0, ax1])
26             T_add[ax0, ax1] = T_matmul_NT[ax0, ax1] + b[ax1]
27
28     @T.prim_func
29     def relu(rxplaceholder: T.Buffer[(1, T.int64(128))], "float32"],
compute: T.Buffer[(1, T.int64(128))], "float32"]) -> None:
30         # function attr dict
31         T.func_attr({"global_symbol": "relu", "tir.noalias": True})
32         # body
33         # with T.block("root")
34         for i0, i1 in T.grid(1, T.int64(128)):
35             with T.block("compute"):
36                 i0_1 = T.axis.spatial(1, i0)
37                 i1_1 = T.axis.spatial(T.int64(128), i1)
38                 T.reads(rxplaceholder[i0_1, i1_1])
39                 T.writes(compute[i0_1, i1_1])
40                 compute[i0_1, i1_1] = T.max(rxplaceholder[i0_1, i1_1],
T.float32(0))
41
42     @T.prim_func
43     def fused_dense_add1(x: T.Buffer[(1, T.int64(128))], "float32"], w:
T.Buffer[(T.int64(10), T.int64(128))], "float32"], b:

```

```

T.Buffer[T.int64(10), "float32"], T_add: T.Buffer[(1, T.int64(10)),
"float32"]) -> None:
44     # function attr dict
45     T.func_attr({"tir.noalias": True, "global_symbol":
"fused_dense_add1"})
46     # body
47     # with T.block("root")
48     T_matmul_NT = T.alloc_buffer([1, T.int64(10)], dtype="float32")
49     for i0, i1, i2 in T.grid(1, T.int64(10), T.int64(128)):
50         with T.block("T_matmul_NT"):
51             i = T.axis.spatial(1, i0)
52             j = T.axis.spatial(T.int64(10), i1)
53             k = T.axis.reduce(T.int64(128), i2)
54             T.reads(x[i, k], w[j, k])
55             T.writes(T_matmul_NT[i, j])
56             with T.init():
57                 T_matmul_NT[i, j] = T.float32(0)
58                 T_matmul_NT[i, j] = T_matmul_NT[i, j] + x[i, k] * w[j,
k]
59     for i0, i1 in T.grid(1, T.int64(10)):
60         with T.block("T_add"):
61             ax0 = T.axis.spatial(1, i0)
62             ax1 = T.axis.spatial(T.int64(10), i1)
63             T.reads(T_matmul_NT[ax0, ax1], b[ax1])
64             T.writes(T_add[ax0, ax1])
65             T_add[ax0, ax1] = T_matmul_NT[ax0, ax1] + b[ax1]
66
67     @R.function
68     def main(x: Tensor((1, 784), "float32")) -> Tensor(None, "float32",
ndim = 2):
69         # block 0
70         with R.dataflow():
71             lv1 = R.call_tir(fused_dense_add0, (x, meta[relay.Constant]
[0], meta[relay.Constant][1]), (1, 128), dtype="float32")
72             lv2 = R.call_tir(relu, (lv1,), (1, 128), dtype="float32")
73             lv4 = R.call_tir(fused_dense_add1, (lv2,
meta[relay.Constant][2], meta[relay.Constant][3]), (1, 10),
dtype="float32")
74             gv: Tensor((1, 10), "float32") = lv4
75             R.output(gv)
76     return gv

```

TVM 编译器可以通过自动图优化手段，在计算图之间进行变换，并在后续的 **Lower** 工作中，将相应的融合反馈到 **TensorIR** 中。接下来又回到了 **TensorIR + Schedule** 变换的主题中。

手工优化一般从计算图融合和优化算子性能两个角度考虑，定制化程度高；编译器的优化思路整体而言和手工优化类似，通过自动化的手段对 IRModule 进行变换（图变换和 schedule 变换），通用化程度较高。

调用外部库

目前已经存在较多高度优化的矩阵计算库如 mkl, cublas, cudnn, cutlass 等被各行各业使用，通过编译器自动优化的方式很难在较短时间内搜索到能够与之匹敌的算法，鉴于此原因，希望将现有的库函数集成到 MLC 的过程中。

首先将已有的库函数注册到 TVM 中：

</>

Python | 收起 ^

```
1 @tvm.register_func("env.linear", override=True)
2 def torch_linear(x: tvm.nd.NDArray,
3                 w: tvm.nd.NDArray,
4                 b: tvm.nd.NDArray,
5                 out: tvm.nd.NDArray):
6     x_torch = torch.from_dlpack(x)
7     w_torch = torch.from_dlpack(w)
8     b_torch = torch.from_dlpack(b)
9     out_torch = torch.from_dlpack(out)
10    torch.mm(x_torch, w_torch.T, out=out_torch)
11    torch.add(out_torch, b_torch, out=out_torch)
12
13 @tvm.register_func("env.relu", override=True)
14 def lnumpy_relu(x: tvm.nd.NDArray,
15               out: tvm.nd.NDArray):
16     x_torch = torch.from_dlpack(x)
17     out_torch = torch.from_dlpack(out)
18     torch.maximum(x_torch, torch.Tensor([0.0]), out=out_torch)
```

只要修改高层 IR 中对计算的描述，调用外部注册的函数即可：

</>

Python | 收起 ^

```
1 @tvm.script.ir_module
2 class MyModuleWithExternCall:
3     @R.function
4     def main(x: Tensor((1, 784), "float32"),
5             w0: Tensor((128, 784), "float32"),
6             b0: Tensor((128,), "float32"),
7             w1: Tensor((10, 128), "float32"),
8             b1: Tensor((10,), "float32")):
```



```
9         # block 0
10         with R.dataflow():
11             lv0 = R.call_tir("env.linear", (x, w0, b0), (1, 128),
dtype="float32")
12             lv1 = R.call_tir("env.relu", (lv0,), (1, 128),
dtype="float32")
13             out = R.call_tir("env.linear", (lv1, w1, b1), (1, 10),
dtype="float32")
14             R.output(out)
15         return out
```

总结

机器学习编译的基本思想同手工优化相似，主要是通过计算图融合和优化单个算子的性能两种方式来提升模型的性能；编译器在计算图融合上更进一步，从过去的特定模板匹配的方式进化成按规则匹配，由处理单个算子的匹配规则到处理一类算子；在算子性能优化上，TVM 编译器采用计算调度分离的方式，通过自动程序优化的手段来寻找更优的算子实现，如果搜索到的算子性能都不理想，也可以通过调用外部库来实现。

手工优化也可以尝试和编译器优化结合来提升模型的性能，例如在编译器优化的基础上，通过手工优化的方式来调整算子的实现；在有先验知识的情况下，可以通过先走指定的图融合策略，后面再去运行编译器自动融合的 pass 来修正图融合的结果等。

References

1. <https://mlc.ai/>
2. <https://tvm.apache.org/docs/arch/index.html>

Inference与CINN结合进行推理

传统的模型手工优化的方法只适用于特定的热点模型，对于各种变种模型常常仍需要投入大量的人力去优化；编译器可以在一定程度上缓解这个问题。我们想要讨论下传统的推理与编译器结合可能的思路，首先看下推理场景一般关注的几个内容（待补充）：

1. **pass**：可维护性好，写起来相对简单；
2. 算子：基本算子和融合算子；
3. 调度开销小；
4. 二次开发能力：用户可定制 pass 和 算子；
5. 子图接入能力：接入TensorRT、oneDNN等高性能引擎；
6. 可裁剪能力：裁剪不需要的第三方库和算子；

- 7. 接口易用和稳定性；
- 8. 多平台支持；
- 9. 硬件接入能力；

我们考虑编译器在以上维度与推理的区别：

1		推理框架	编译器	
2	pass			比较相通的概念，不管在传统自身IR体系的pass抽象，但不同，推理中的pass绝大多量化，但编译器中的pass绝大的一部分；传统框架的IR和编程化的实现问题；
3	算子		编译器生成融合算子的能力较强，只要拼接算子IR表示即可	
4	调度开销		理论上来说编译器能做到纯算子运行模式，框架调度为0	直观感受上来看，框架调度-（需测试）
5	二次开发能力			无明显区别
6	子图接入能力			无明显区别
7	可裁剪能力		编译器的强项所在	
8	接口易用和稳定性			理论上无明显区别
9	多平台支持			无明显区别
10	硬件接入能力		编译器的核心优势	

CINN目前相对缺失的工作（了解有限）

对CINN了解有限，列出的不一定对：

- 目前不支持动态Shape（难度较高）；
- 算子相对有限：需补充OpMapper进行大算子拆分；
- Schedule自动优化开发中；
- 未验证多线程能力；

思路一：基于CINN做推理

依赖：CINN足够完善

理想状态：完全依赖CINN进行推理，CINN 编译器的能力作为baseline，手工优化通过添加规则（特定融合pass）和算子（手工优化的高性能算子）进行调优；

这是一种比较理想和激进的状态，完全依赖编译器框架进行推理，我们开发者主要是将通过框架的二次开发能力对热点模型进行手工优化（pass + 算子），短期内难以达到这个目标。

思路二：编译器只用来做代码生成

将 CINN 或 TVM 编译器的运行时剥离，仅使用其代码生成的能力；开发者仅需开发定制的 Pass，推理框架将选定的算子交给编译器，编译器返回schedule优化后的函数；

思路三：编译器作为子图引擎

现有状态：CINN或TVM编译器作为第三方库集成到推理框架中，现在实现也是这么做的，但不够灵活（实现类似于集成trt，能进编译器的算子都会进编译器），很难说这是一种比较好的方式。